

Teaching an Industrial Manipulator: Offline Programming in FANUC RoboGuide and Transfer to a Physical Robot Cell

Prajjwal Dutta

Ira A. Fulton Schools of Engineering
Arizona State University
Tempe, AZ, USA
pdutta9@asu.edu

Abstract—This report documents a sequence of hands-on experiments in industrial robot programming carried out as part of an Industry 4.0 laboratory module. The work covers the full path from an offline simulated cell to a physical FANUC manipulator: building a robot cell in FANUC RoboGuide, operating the virtual teach pendant, writing and stepping through motion programs, importing CAD fixtures, adding register-driven loops, and finally establishing coordinate frames on real hardware using the three-point method. Two programs were taught on the physical robot. The first traces a rectangular path twice, once with joint interpolation and once with linear interpolation, expressed in the World frame. The second re-expresses the same rectangular path in a taught User frame. The measured user frame origin and orientation are reported. The exercise shows that the same taught path behaves differently under joint and linear interpolation, and that a correctly taught user frame allows a program to be written in coordinates that are meaningful to the workpiece rather than to the robot base.

Index Terms—industrial robotics, FANUC, RoboGuide, offline programming, teach pendant, coordinate frames, Industry 4.0

I. INTRODUCTION

Industrial manipulators are still predominantly programmed by teaching: an operator jogs the robot to a pose, records it, and assembles the recorded poses into a motion program. This remains true even as offline programming and simulation tools mature, because a simulated cell is only useful to the extent that the poses it produces can be transferred onto hardware whose geometry never matches the model exactly.

This report documents a laboratory module intended to make that transfer concrete. The module is structured as six graded exercises, beginning with cell construction in FANUC RoboGuide and ending with frame setup on a physical FANUC robot. The objective of the work reported here was not to produce a novel algorithm but to build operational fluency with the tools an industrial integrator actually uses, and to observe directly where simulation and hardware agree and where they do not.

The contributions of this report are (i) a record of the six exercises as performed, (ii) the two motion programs as they were taught on hardware, transcribed from the teach pendant, and (iii) the measured parameters of a user frame established by the three-point method, together with an account of how

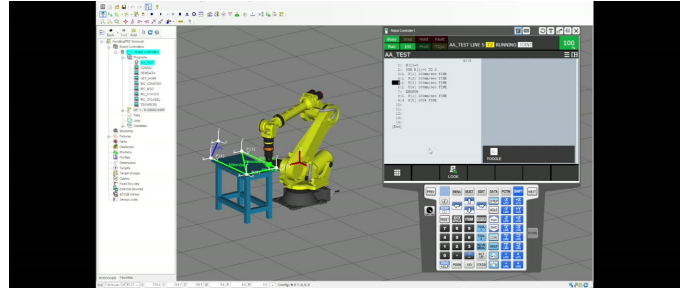


Fig. 1. The RoboGuide cell during a looped trace of the table edges. The virtual teach pendant is docked at the right of the workspace view and shows the program executing line by line.

that frame changes the interpretation of an otherwise identical program.

II. EXPERIMENTAL SETUP

A. Simulated Cell

The offline work was performed in FANUC RoboGuide. A new cell was created using the default configuration presented by the cell creation wizard, accepting the default selection at each of the eight steps and selecting flange type 1. The resulting workcell contains a six-axis manipulator, giving three translational degrees of freedom for position and three rotational degrees of freedom for orientation, together with a virtual teach pendant that reproduces the key layout and menu structure of the physical pendant.

Fig. 1 shows the simulated cell after a table fixture had been imported from the RoboGuide CAD library and aligned within the reachable workspace of the manipulator.

B. Physical Cell

The hardware exercises were performed on a six-axis FANUC manipulator installed in a guarded laboratory cell. The robot carries a dressed tool assembly with an integrated illuminator and a vacuum cup, and works over a fixtured sheet-metal plate. Fig. 2 shows the cell during teaching and Fig. 3 shows the tool over the fixture.

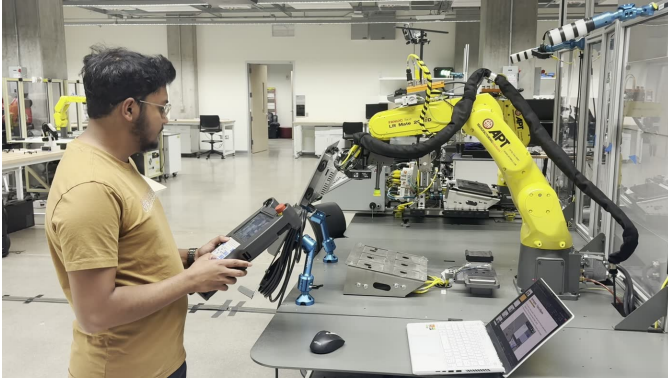


Fig. 2. Teaching the physical robot from the pendant inside the guarded cell.



Fig. 3. The tool assembly over the fixtured plate. The tool carries an illuminator and a vacuum cup on an extended post, which is the feature used as the tool centre point reference when jogging to the frame teaching positions.

All hardware teaching and execution was performed in T1 mode with the speed override set to 50%, which is the appropriate configuration for manual operation with personnel inside the guarded area. The teach pendant enable device must be held for motion to be permitted; releasing it raises SRVO-003 Deadman switch released and aborts the running program. This condition is visible in the program screenshots reproduced later in this report, because the enable device was released in order to photograph the pendant.

III. METHOD

A. Interface and Tool Centre Point

The first two exercises established the operating interface. The workspace view was manipulated to inspect the manipulator from several angles and the reachable workspace envelope was displayed. The tool centre point (TCP) was then exercised directly: dragging the TCP marker along a displayed axis translates the tool, and dragging while holding shift rotates the tool about that axis. The same poses can be entered numerically through the position screen on the pendant, which

was used whenever a repeatable pose was required rather than an approximate one.

B. First Program: Rectangular Trace

A program was created from the pendant using the keyboard entry option. Two pendant behaviours are prerequisites for any program work and were exercised first. A fault condition blocks execution until it is cleared, which is done by holding shift and pressing reset. Single-line execution is obtained by holding shift and pressing the step key, which is the safe way to verify a newly taught path before running it continuously.

Four positions were then recorded to trace a rectangle. Positions were reached either by dragging the TCP or by entering coordinates numerically, and each was committed with the add move point function, which records the pose at the time of the instruction. The program was executed by holding shift and pressing the forward key with the cursor at the first line.

C. Fixtures and Looped Motion

A table was imported from the CAD library and aligned so that it lay within the reachable workspace. Four positions were recorded on the table edges. To repeat the trace, a numeric register was introduced as a loop counter and a FOR/ENDFOR construct was added around the four motion instructions, configured for five iterations, followed by a move to a position above the centre of the table serving as a home pose. This is the program shown executing in Fig. 1.

D. Coordinate Frames

The fourth exercise compared the available jogging frames. In the World frame the first three jog keys translate the TCP along fixed Cartesian axes and the remaining three rotate it. In the Joint frame each key drives one of the six axes directly. The User and Tool frames re-express the same Cartesian jogging relative to a taught workpiece frame or to the tool itself. The active frame is selected by deselecting shift and pressing the coordinate key.

E. Three-Point User Frame

The final exercise established a user frame on hardware. A fixture was placed in the cell and a new user frame was created from the setup menu, with the three-point method selected. The method requires three taught poses:

- 1) the frame origin, taught at the bottom left corner of the jig;
- 2) a point in the $+X$ direction, taught at the bottom right corner;
- 3) a point in the $+Y$ direction, taught at the top left corner.

Each was reached by jogging the TCP to the corner and committing the pose with shift and record. The frame was then activated by setting the active user frame number to 1, after which the displayed TCP axes reorient to the taught frame and Cartesian jogging follows the jig rather than the robot base.

TABLE I
PROGRAM PROG1 AS TAUGHT, WORLD FRAME

Line	Instruction	Termination
1	J @P[1] 100%	FINE
2	J P[2] 100%	FINE
3	J P[3] 100%	FINE
4	J P[4] 100%	FINE
5	J @P[1] 100%	FINE
6	L P[5] 100mm/sec	FINE
7	L P[6] 100mm/sec	FINE
8	L P[7] 100mm/sec	FINE
9	L P[8] 100mm/sec	FINE
10	L P[5] 100mm/sec	FINE
11	J @P[1] 100%	FINE

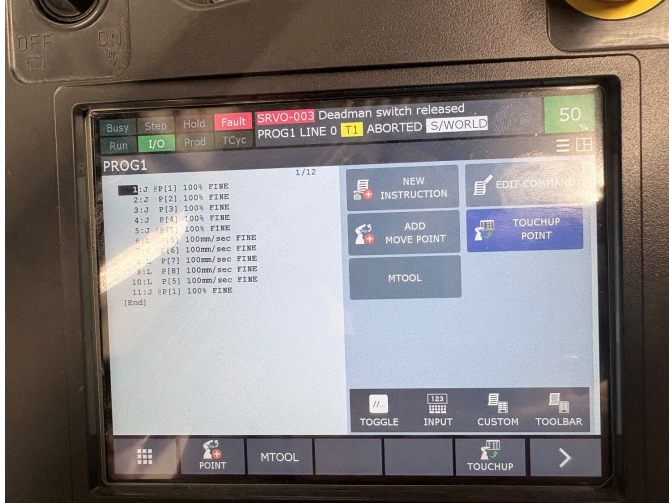


Fig. 4. PROG1 on the teach pendant. The status line shows T1 mode, 50% override and the World frame active.

IV. RESULTS

A. Taught Programs

Table I reproduces the first program as taught on hardware, transcribed from Fig. 4. The program traces a rectangle twice. Lines 1 to 5 use joint interpolation at 100% speed through P[1] to P[4] and return to P[1]. Lines 6 to 10 use linear interpolation at 100 mm/sec through P[5] to P[8] and return to P[5]. Line 11 returns to P[1]. Every instruction uses FINE termination, so the robot settles on each taught pose rather than blending through it. The status line reports the program aborted at line 0 in T1 mode with the World frame active (S/WORLD).

Table II reproduces the second program, transcribed from Fig. 5. It retains the linear segment of the rectangle but is executed with the taught user frame active, indicated by S/USER on the status line in place of S/WORLD.

B. Measured User Frame

Table III gives the user frame as recorded by the controller after the three teaching points were taken. All three constituent points are reported as USED, confirming that the controller

TABLE II
PROGRAM PROG2 AS TAUGHT, USER FRAME 1 ACTIVE

Line	Instruction	Termination
1	J @P[1] 100%	FINE
2	L P[5] 100mm/sec	FINE
3	L P[6] 100mm/sec	FINE
4	L P[7] 100mm/sec	FINE
5	L @P[1] 100mm/sec	FINE



Fig. 5. PROG2 on the teach pendant, with User frame 1 active.

TABLE III
USER FRAME 1, THREE-POINT METHOD

Component	Value	Component	Value
X	401.4	W	9.4
Y	144.1	P	1.7
Z	-76.9	R	-87.5
Orient Origin Point: USED			
X Direction Point: USED			
Y Direction Point: USED			

accepted the taught triad and computed a frame from it rather than falling back on a default.

The R value of -87.5° is the substantive result. It states that the jig was seated in the cell rotated by very nearly a right angle with respect to the robot base, while W and P remain within a couple of degrees of zero, indicating that the jig surface was close to level. Without the taught frame, every position on that jig would have to be expressed in base coordinates and would carry that rotation implicitly.

C. Observed Behaviour

Three observations follow directly from running the programs.

First, joint and linear interpolation between the same pair of taught poses do not produce the same path. Under joint interpolation each axis is driven from its start value to its end value independently and the TCP sweeps an arc. Under linear

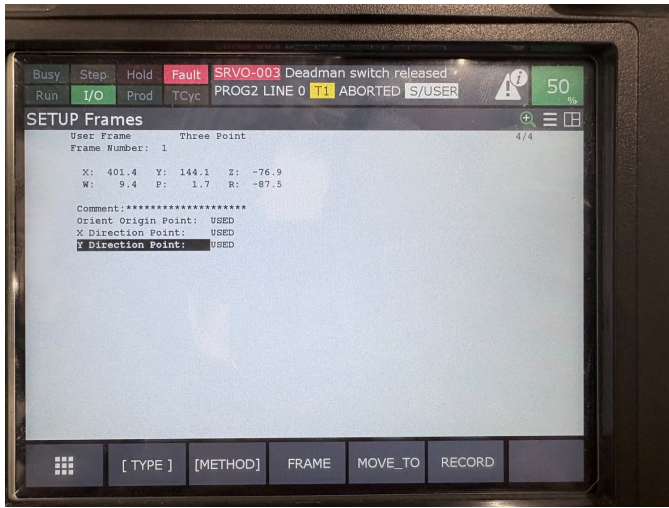


Fig. 6. The frame setup screen after teaching. Translations are in millimetres and the W , P , R triple gives the frame orientation in degrees.

interpolation the controller holds the TCP on the straight segment between the poses. The rectangle is only geometrically a rectangle in the linear segment of `PROG1`; the joint segment visits the same four corners but does not travel along the edges between them.

Second, `FINE` termination is what makes the taught corners visible. The robot decelerates to each pose before starting the next instruction, so the traced figure has sharp corners at the cost of cycle time.

Third, activating the user frame changes the meaning of the jog keys and of any subsequently taught position, but does not retroactively move positions already recorded in another frame. This is the practical reason the frame is taught before the positions that depend on it.

V. DISCUSSION

The module is deliberately arranged so that each exercise supplies a prerequisite for the next: the interface before the program, the program before the loop, the frames before the frame setup. The ordering matters because the failure modes are cumulative. A program taught before a fault is understood cannot be run; a looped program written without a register has nothing to count with; a position taught before the user frame is active is recorded in the wrong frame and will appear displaced once the frame is activated.

The transfer from RoboGuide to hardware is where the exercise earns its place. In simulation a pose can be entered numerically and is exact by construction. On hardware the same pose has to be reached by jogging, and the achievable precision is limited by how accurately the operator can align the tool with a physical corner. The three-point method is a direct response to that limit: it does not ask for exact coordinates, only for three physically identifiable points on the workpiece, and derives the frame from them.

The principal limitation of this work is that it reports a taught frame and two taught programs without an independent

measurement of the resulting path accuracy. Establishing how closely the executed path follows the intended rectangle would require external metrology, which was outside the scope of the module. A natural extension would be to repeat the frame teaching several times and report the spread in the resulting frame parameters, which would quantify how much of the positioning error is attributable to the operator rather than to the manipulator.

VI. CONCLUSION

Six exercises were completed, moving from cell construction and pendant operation in FANUC RoboGuide through program creation, CAD fixtures and register-driven loops, and concluding with coordinate frame setup on a physical FANUC manipulator. Two programs were taught on hardware and are reported here as transcribed from the pendant, together with the parameters of a user frame established by the three-point method. The measured frame shows the jig seated at close to -90° about the vertical with respect to the robot base, which is precisely the offset that a taught user frame exists to absorb. The exercise supports a simple conclusion about industrial robot programming: the difficulty lies less in specifying motion than in establishing the frames that make the specification mean what the programmer intends.

ACKNOWLEDGMENT

The laboratory module, including the exercise structure and the accompanying instructional material, was developed and delivered by Rajesh S. Aouti, Dr. Cynthia Pickering and Dr. Sangram Redkar as part of the PSMART hands-on session series at Arizona State University. The RoboGuide cell configuration and the sequence of exercises follow that material.

REFERENCES

- [1] R. S. Aouti, C. Pickering and S. Redkar, "PSMART Session 2: FANUC RoboGuide hands-on sessions," instructional material, Arizona State University, 2026.
- [2] FANUC America Corporation, *ROBOGUIDE Simulation Software*, product documentation, Rochester Hills, MI.
- [3] FANUC America Corporation, *R-30iB Plus Controller Operator's Manual*, Rochester Hills, MI.
- [4] J. Craig, *Introduction to Robotics: Mechanics and Control*, 4th ed. Harlow, UK: Pearson, 2018.